

03b_extension_integration

HXC-159 Phase 1a — Constitution-derived extension integration matrix

Field	Value
Revision	1
Created	2026-07-29
Last modified	2026-07-29
Status	Complete — 21 extensions classified, 0 silent omissions; 3 live defects found
Workable item	HXC-159 (Task, Queued, High)
Mandate	Operator (2026-07-29): <i>“Make sure that SpecKit and Superpowers are used with Bridge extension and with all extensions we have created derived from constitution Submodule!”</i>
Governing anchors	§11.4.74 (extend-don’t-reimplement) · §11.4.164 (auto-propagation) · §11.4.228 (inherit by reference, never copy) · §11.4.118 (enumerated negatives are evidence)

Table of contents

- [1. Scope and method](#)
- [2. The registration path HelixSkills must compose with](#)
- [3. LIVE DEFECTS found while establishing the baseline](#)
- [4. Matrix — constitution skills \(7\)](#)
- [5. Matrix — constitution MCP servers \(2\)](#)
- [6. Matrix — constitution plugins \(2\)](#)
- [7. Matrix — constitution actions \(2\)](#)
- [8. Matrix — constitution hooks \(7 live\)](#)
- [9. Matrix — SpecKit, Superpowers, Bridge](#)
- [10. Summary tally](#)
- [11. Honest boundary](#)

1. Scope and method

Every constitution-derived extension named by the operator is classified against HelixSkills with exactly one verdict from a closed set:

Verdict	Meaning
REUSE	HelixSkills consumes it as-is; no change to the extension.
EXTEND	§11.4.74 applies — the capability is 80%+ there; the missing part is added to the constitution , never duplicated in HelixCode.
COMPOSE	Both systems own part of one workflow; the integration is a defined hand-off, not a merge.
ORTHOGONAL	No interaction. An explicit reason is still recorded — per §11.4.118, silence is not an answer.

21 extensions are classified. None is omitted.

2. The registration path HelixSkills must compose with

constitution/scripts/post_update_hook.sh (20 975 bytes, executable) is the §11.4.164 auto-propagation engine. Its main calls exactly four installers, at :543-546:

```
install_skills           (:199)  – CHANGED_SKILLS → <project>/skills/<name>
install_mcp_configs      (:255)  – constitution/mcp/*.json merged into the
install_hooks            (:319)  – constitution/scripts/hooks/* into .git/
install_action_plugins   (:359)  – constitution/plugins/* registered as ag
```

Change detection is by path prefix at :146-159 (skills/*, mcp/*.json, hook basenames).

Two destinations, not one. install_skills links constitution/skills/<name> → <project>/skills/<name> (:208-231), then runs that skill's register.sh (:236-240), which delegates to install_cli_agent_plugins.sh

and produces the **second** link `<project>/ .claude/skills/<name>`. The installer targets **four** agent formats — `install_cli_agent_plugins.sh:158:`

```
local specs="gemini:.gemini/commands:toml qwen:.qwen/commands:toml codex:p
plus .claude/skills/ symlinks (:111, :145) and a marketplace + plugin install
(:41).
```

Data-safety posture is already correct — `install_skills:225-233` only replaces its own symlink or a free slot and explicitly refuses to `rm -rf` a real directory, citing §9.2 / §11.4.122. So it will **not** clobber SpecKit's `.claude/skills/ speckit-*` directories.

2.1 The composition requirement, stated concretely

<project>/skills/ — the directory `post_update_hook.sh` installs INTO — is character-for-character the directory `helix_agent`'s skill registry reads FROM.

- `helix_agent/internal/router/router.go:333` —
 `skillConfig.SkillsDirectory = "skills"`
- `post_update_hook.sh:207` — `local project_skills_dir="$ {PROJECT_ROOT}/skills"`

This is the single highest-value integration fact in this document. The constitution's skill distribution path and `helix_agent`'s skill discovery path **already point at the same directory** — they simply have never been pointed at the same *project root* at the same time. A HelixSkills incorporation that establishes a *third*, competing registration path would be a §11.4.74 violation when a two-line convergence is available.

Blocking caveat (§5.1 of 03): the two ends disagree on schema.

`post_update_hook` installs constitution SKILL.md files whose frontmatter is `name/description/version`; `helix_agent`'s parser reads that shape, but `helix_code`'s parser needs `triggers/variables/requires_isolation`. Convergence on the directory does not by itself give convergence on the format.

2.2 What `post_update_hook` does NOT know about

```
$ grep -niE 'speckit|superpower|superspec|superb|bridge' constitution/scri
(no output)
```

The §11.4.164 auto-propagation engine has zero SpecKit, Superpowers or Bridge awareness. A constitution pull registers skills, MCP configs, hooks and action-plugins — and leaves SpecKit's `.specify/` and `.claude/skills/ speckit-*` entirely unmanaged. Combined with 01 §9 (the Bridge's own installers

do not exist), **nothing in the constitution currently installs, updates, validates or even notices the SpecKit/Superpowers layer.** That is the gap the operator’s requirement names, and it is unclaimed work.

3. LIVE DEFECTS found while establishing the baseline

These were found by probing, not by inspection of intent. Each is independently actionable.

3.1 Registration census: 1 of 7 constitution skills is registered, and that one is broken

SKILL	root skills/	.claude/skills/
action-prefix-system	absent	absent
media-validator	LINKED-BUT-BROKEN	absent
multitrack	absent	absent
reporting-workable-items	absent	absent
scheduled-work-queue	absent	absent
session-sync	absent	absent
workable-item-lifecycle	absent	absent

Not one constitution skill is registered into .claude/skills/ in this checkout. The machinery is present and executable (`post_update_hook.sh`, `install_cli_agent_plugins.sh`, and a working `register.sh` per skill) — it has simply never completed here.

3.2 The single existing symlink is dangling, and points at a foreign OS

```
$ ls -la skills/media-validator
skills/media-validator -> /Volumes/T7/Projects/helix_code/constitution/ski

$ [ -e skills/media-validator ] && echo VALID || echo BROKEN
BROKEN
```

```
$ ls -ld /Volumes/T7/Projects/helix_code/constitution/skills/media-validato
ls: cannot access '...': No such file or directory
```

`/Volumes/T7/...` is a **macOS** mount path, resolved on a **Linux** host. This symlink was created by a `post_update_hook` run on a different machine and committed or carried across. It is a cross-platform absolute-path leak — an

§11.4.81 (cross-platform parity) and §11.4.111 (resolve-by-stable-name, not by machine-specific absolute path) concern.

3.3 helix_agent ships 1 174 skills that the constitution’s installer would collide with

submodules/helix_agent/skills/ already exists with 15 vendor namespaces (azure, claude-code, codex, data, development, devops, forge, github-copilot, gptme, misc, openhands, plugins, postgres-mcp, ui-ux, web). If post_update_hook.sh were ever run with PROJECT_ROOT=submodules/helix_agent, it would write constitution symlinks into that same skills/ tree, mixing two schemas in one directory that a single flat-scanning parser (Registry.Load, registry.go:73) reads indiscriminately.

Not a live break today (the hook has not been run there), but it is a designed-in collision that Phase 2 must prevent — most cleanly by namespacing constitution skills under skills/constitution/<name>.

4. Matrix — constitution skills (7)

All seven live at constitution/skills/<name>/ as SKILL.md + register.sh + .html/.pdf/.docx. Frontmatter schema (verified on action-prefix-system/SKILL.md:1-5): name, description, version.

#	Skill	Verdict	Interaction with HelixSkills
1	action-prefix-system	COMPOSE	Owns §11.4.140 grammar (6 forms) and the registry.yaml lookup. HelixSkills’ skill_create / skill_search MCP tools (Ø2 §8.1) are natural targets for new registered actions, and the FEATURE directive (§11.4.213) is how a skill-graph research task gets scheduled. Hand-off, not merge: the action system resolves <i>prompt grammar</i> ; HelixSkills resolves <i>skill identity</i> . Neither should absorb the other.
2	media-validator	ORTHOGONAL	§11.4.163 media-artifact validation (OCR/metadata/pattern → PASS/FAIL). HelixSkills produces no media artifacts — its evidence is DB rows, JSON/TOML payloads and text. Reason recorded, not silence: it becomes relevant only if a later phase adds §11.4.158 recorded-video evidence for skill-graph features, at which point it is REUSE with no change.
3	multitrack	ORTHOGONAL	§11.4.187 parallel-track orchestration — governs <i>how work is executed across tracks</i> , entirely above the skill-

#	Skill	Verdict	Interaction with HelixSkills
			data layer. HelixSkills is a subject of multitrack work, never a participant in it. Sole coupling: any HelixSkills work must carry §11.4.182 track labels, which is a process obligation on the <i>agent</i> , not an integration surface.
4	reporting-workable-items	COMPOSE	Owens the ISSUE/BUG/TASK engine (§11.4.202) writing to <code>docs/workable_items.db</code> . HXC-159 itself is one of its rows. Each defect in §3 above, and each in 03 §7, should become an item through this skill — not through a HelixSkills-native tracker. HelixSkills has <i>no</i> item tracker, so there is no contention.
5	scheduled-work-queue	COMPOSE	Durable BACKGROUND/REMINDER queue. Directly relevant: HelixSkills ships 6 worker job types (autoexpand, validate, codeanalysis, registry_review, batch_embed, source_rescan — 02 §8.4) driven by <code>robfig/cron</code> . Two schedulers would be a §11.4.74 duplication. Phase-2 decision required: does HelixSkills' cron worker remain internal (it is a <i>service</i> concern, cron-in-process), or do its jobs surface as scheduled-work-queue entries (agent-visible, durable, operator-inspectable)? I recommend the former for in-service jobs and the latter for any agent-triggered re-sync — but this is a decision, not a finding.
6	session-sync	ORTHOGONAL	§11.4.207/§11.4.131 session-resumption state. Concerns <i>agent session continuity</i> ; HelixSkills state is server-side in PostgreSQL and survives sessions by construction. Reason recorded: the only conceivable overlap — a resumption prompt citing skill-graph state — is a content question for the session-sync skill, requiring nothing from HelixSkills.
7	workable-item-lifecycle	COMPOSE	Status/type/id integrity (§11.4.148), closure vocabulary (§11.4.33), reopen attribution (§11.4.34). Governs how HXC-159 and its children are opened, statused and closed. Note the conceptual near-collision : HelixSkills has its own lifecycle enum <code>SkillStatus{draft,validated,active,deprecated}</code> (models/skill.go:12-19) which is about <i>skills</i> , and this skill governs <i>work items</i> . They must not be conflated — different nouns, similar shape.

5. Matrix — constitution MCP servers (2)

#	MCP server	Verdict	Interaction
8	media-validator-mcp.json	ORTHOGONAL	Same reasoning as #2. Verified config: "command": "bash", "args": ["skills/media-validator/media-validator.sh"], "env": {"MEDIA_VALIDATOR_EVIDENCE_DIR": "qa-results/media-validator"}. Note the arg is relative to skills/ — so it depends on the §3.1 registration that is currently broken; the MCP server would fail to start today.
9	scheduled-work-mcp.json	COMPOSE	The MCP face of #5. If HelixSkills' worker jobs are ever agent-triggered, this is the existing tool surface to extend rather than adding HelixSkills-native scheduling tools. Composes with HelixSkills' own 13-tool MCP server (02 §8.1) as a sibling server , not a competitor — an agent can hold both.

Structural note for both: the constitution's MCP surface is **config-level** (JSON specs merged into the agent's MCP config by `install_mcp_configs:255`). HelixSkills' MCP surface is a **compiled Go server** (`internal/mcp/server.go:54`) with stdio, HTTP and ACP transports. These are different layers of the same protocol and do not conflict: HelixSkills would be registered as one more entry in exactly the config that `install_mcp_configs` manages. **That is the cleanest cross-consumer integration path in the entire analysis** — it requires no Go coupling in either consumer (see 03 §6 rank 8) and works identically for `helix_code` and `helix_agent`.

6. Matrix — constitution plugins (2)

#	Plugin	Verdict	Interaction
10	helix	COMPOSE	<code>plugin.json</code> verbatim: <i>“Helix Constitution action directives as native slash commands... Commands are GENERATED from actions/registry.yaml — adding an action is a data</i>

#	Plugin	Verdict	Interaction
			<i>change, never a code change.”</i> This is the delivery vehicle for #1 and is how the helix: * skills reach this session today . Registered via install_action_plugins:359. HelixSkills would compose by contributing registered actions, never by shipping a competing slash-command plugin.
11	scheduled-work	COMPOSE	Contains .mcp.json, build.sh, .claude-plugin/plugin.json — the packaged form of #5/#9. Same verdict, same reasoning. Its build.sh means it is a compiled plugin, so any HelixSkills contribution to it is a code change to the constitution (§11.4.74 EXTEND path), not a config change.

7. Matrix — constitution actions (2)

#	Artifact	Verdict	Interaction
12	actions/registry.yaml	EXTEND	The §11.4.140 single source of truth. Verified schema: schema_version: 1, a grammar: block defining all six forms (prefix_regex:44, colon_form_regex:45, single_colon_form_regex:57, slash_form_regex:61, arrow_form_regex:63, plus namespace_separator, body_separator, multiple_prefixes: stack, escape: '\ '), then an actions: list (:74) with per-action name/version/namespaces/slash_bare/slash_conflicts/summary/

#	Artifact	Verdict	Interaction
			expansion/rules. The only genuine EXTEND in this matrix: if HelixSkills warrants operator-facing directives (e.g. a SKILL :: directive to create or query a graph skill), they are added as rows here — a data change to the constitution, never a new grammar in HelixCode. The file's own design ("adding an action is a data change, never a code change") makes this the cheapest possible extension path.
13	actions/ subagent_tiering.yaml	ORTHOGONAL	§11.4.231 nano-precision model-tier selection — governs which model tier a dispatched subagent runs on. A concern of the <i>dispatching agent</i> , with no data or control-flow relationship to a skill graph. Reason recorded: the only touchpoint is that HelixSkills incorporation work should itself be tiered per this file — a process obligation on me, not an integration surface.

8. Matrix — constitution hooks (7 live)

All at constitution/scripts/hooks/, installed by `install_hooks:319`. The directory also holds 4 `test_*.sh` files (their paired tests), not counted as separate extensions.

#	Hook	Verdict	Interaction
14	action_prefix_expand.sh	COMPOSE	Runtime expander for #1/#12. Same hand-off.
15	credential_scan_lib.sh	REUSE	CONST-042 / §11.4.10 secret-leak scanning. Directly load-bearing: HelixSkills handles GitHub tokens (<code>github.TokenFromEnv</code> , 02

#	Hook	Verdict	Interaction
			§6.E6), LLM provider keys (autoexpand.NewLLMClientFromConfig), a Postgres DSN and an API-key auth middleware (api.APIKeyAuth). Its config uses <code>\${VAR}</code> interpolation (config.go:28) precisely so secrets stay out of files. Any incorporation reuses this scanner unchanged — it is the existing enforcement for a risk HelixSkills genuinely carries.
16	guard-forbidden-commands.sh	REUSE	§11.4.109 PreToolUse guard (host-direct emulator, force-push, sudo, host-power). Applies to HelixSkills work as to all work. Specifically relevant: HelixSkills ships <code>deploy/docker-compose.yml</code> and 18 ops scripts including <code>start.sh/stop.sh/restart.sh</code> ; §11.4.161 requires rootless podman, and this guard is the mechanical enforcement. Reused unchanged.
17	guard-track-branch-label.sh	REUSE	§11.4.182 label enforcement. Empirically verified in this very session — it blocked two of my subagent dispatches until the (T1/main - claude1 - opus - xhigh) prefix was supplied, and is the only wired PreToolUse hook in <code>.claude/settings.json</code> . Reused unchanged.
18	guard-branch-consistency.sh	REUSE	§11.4.181 one-feature-one-branch-name. Applies when HXC-159 work lands on a feature branch. Reused unchanged.
19	guard-work-track-binding.sh	REUSE	§11.4.191 work-to-track binding via <code>logic_groups.destination + group_paths</code> . Directly relevant: if HelixSkills lands as a submodule, its paths must be bound to a logic group or this guard will (correctly) block commits. A Phase-2 registration task, not a code change.
20	post-merge	COMPOSE	The §11.4.164 trigger that fires <code>post_update_hook.sh</code> after a constitution pull. This is the exact seam §2 discusses: whatever registration path HelixSkills uses must be reachable from

#	Hook	Verdict	Interaction
			here, or it will silently drift out of sync on every constitution update — the §11.4.86 drift class.

9. Matrix — SpecKit, Superpowers, Bridge

#	Extension	Present?	Verdict	Interaction
21a	SpecKit (L2)	YES — installed by this task	REUSE	Drives all HXC-159 phases per operator mandate: /speckit-specify → /speckit-plan → /speckit-tasks → /speckit-implement. 10 skills + .specify/ (Ø1 §6). Note the co-tenancy in .claude/skills/ (Ø1 §7.2) — §2 confirms install_skills will not clobber it.
21b	Superpowers	YES — via Claude Code plugin system	REUSE	superpowers:brainstorming, systematic-debugging (§11.4.102), test-driven-development (§11.4.43/ §11.4.115), subagent-driven-development (§11.4.70), writing-plans, requesting-code-review (§11.4.125/ §11.4.142) are all live in this session. Sourced from ~/.claude-claude1/plugins/installed_plugins.json, not from constitution/submodules/superpowers (which does not exist).
21c	Bridge (L3 SuperSpec / L4 SuperB / L5 SuperBridge MCP)	NO — design only	BLOCKED	Every probe negative: superspec not on PATH; specify extension list → “No extensions installed.”; superbridge not on PATH; no reference in .mcp.json or .claude/settings*.json;

#	Extension	Present?	Verdict	Interaction
				constitution/scripts/extensions/ does not exist, so neither Bridge install script exists ; constitution/submodules/ contains none of the four expected submodules; constitution/.gitmodules has no matching entry. Full analysis + layer table in Ø1 §9.

On the operator’s requirement. It asks that SpecKit and Superpowers be *used with* the Bridge extension. SpecKit is installed and Superpowers is available, so two of the three named components are satisfiable today. **The Bridge is not — it cannot be “used” because it is not installed, and it cannot be installed because its installer has never been written.** Reporting otherwise would be exactly the §11.4.108 SOURCE-vs-RUNTIME bluff this requirement warns against.

Recommended disposition: raise “author install_speckit_superpowers_bridge.sh + validate_bridge.sh and wire L3/L4/L5” as its own tracked item via extension #4 (reporting-workable-items). HXC-159 Phase 2 then either depends on that item or explicitly scopes around it — but must not assume the Bridge.

10. Summary tally

Verdict	Count	Members
REUSE	7	credential_scan_lib, guard-forbidden-commands, guard-track-branch-label, guard-branch-consistency, guard-work-track-binding, SpecKit, Superpowers
EXTEND	1	actions/registry.yaml
COMPOSE	9	action-prefix-system, reporting-workable-items, scheduled-work-queue, workable-item-lifecycle, scheduled-

Verdict	Count	Members
		work-mcp.json, helix plugin, scheduled-work plugin, action_prefix_expand.sh, post-merge
ORTHOGONAL	5	media-validator, multitrack, session-sync, media-validator-mcp.json, subagent_tiering.yaml
BLOCKED	1	Bridge (L3/L4/L5)
Total	23	(21 numbered rows; row 21 splits into 3)

Every ORTHOGONAL verdict carries an explicit reason above — none is a silent omission (§11.4.118).

Only one true EXTEND. That is the healthy outcome: the constitution’s extensions and HelixSkills occupy genuinely different layers, so the integration is overwhelmingly *composition* and *reuse* rather than modification. The §11.4.74 duplication risk concentrates in exactly one place — **scheduling** (#5/#9/#11 vs HelixSkills’ 6 cron job types) — and that is a Phase-2 decision, flagged not decided.

11. Honest boundary

Verified by captured probe in this session: the registration census (§3.1) and the dangling symlink (§3.2); `post_update_hook.sh`’s four installers and their line numbers; that it contains no `speckit/superpowers/bridge` reference; that `install_skills` targets `<project>/skills/` and refuses to delete a real directory; that `<project>/skills/` equals `helix_agent`’s configured `SkillsDirectory`; the four-format target list in `install_cli_agent_plugins.sh:158`; the `registry.yaml` grammar and action schema; both plugin manifests; the full contents of `constitution/{skills,mcp,plugins,actions,scripts/hooks}`; and every negative Bridge probe in §9.

Not verified: - **The verdicts themselves are judgments, not measurements.** Each is argued from what the extension does and what HelixSkills does, but no integration was built or tested. A Phase-2 design pass may reclassify any row — particularly #5/#9/#11 (scheduling), where I have deliberately flagged a decision rather than made one. - I read `SKILL.md` in full for **one** of seven constitution skills

(action-prefix-system); for the other six I relied on file listings, the generated Layer-3 catalogue entries in the upstream repo, and their constitutional anchors. A full read could shift an ORTHOGONAL to a COMPOSE. - I read `post_update_hook.sh` selectively (installer signatures, `install_skills` body, change detection, main dispatch) — roughly 120 of 20 975 bytes. Behaviour outside those regions is not characterised. - I did **not** run `post_update_hook.sh` or any `register.sh`. §3.1 reports the *current* state; it does not prove the machinery would succeed if run. Fixing §3.1/§3.2 is a separate action requiring its own evidence, and is deliberately not attempted here — this phase is inventory, and the registration state belongs to shared paths other live streams may touch. - Six of the eight Bridge documents (NANO_TASK_ENGINE, EXTENSION_DEVELOPMENT, TDD_INTEGRATION, CONSTITUTION_INTEGRATION, SECURITY, APPENDIX) were **not** read. Obligations stated only there are not reflected in row 21c.